Started on	Monday, 16 January 2023, 8:06 AM
State	Finished
Completed on	Monday, 16 January 2023, 9:43 AM
Time taken	1 hour 36 mins
Grade	Not yet graded

Question 1

Complete

Marked out of 1.00

Flag question

0.Setup

Download the clean src.

1. Delete the old src folder in HAL9000 folder (or move it somewhere else) and replace it with the new src.

2. Open HAL9000.sln in Visual Studio, select **Userprog** config and rebuild the HAL9000 solution.

Threads (4p)

1. (1p) Modify the way in which thread ID allocation works, such that the TIDs are allocated in the following way: the first thread will have as TID the day of your birthday. The TID for the consecutive threads will be TID of the previous thread PLUS the month of your birthday.

Example: If you were born on the 14th of January, the day of your birthday is 14 and the month of your birthday is 1. Thread IDs should be 14,15,16 etc.

Display at thread creation the message "Thread [tid = 0x%X] is being created".

2. (1.5p) Add a field in the THREAD structure to keep for each thread the TID of the thread that created it (parent of a thread). Display it in *CmdListThreads()* function, for each thread.

3. (1.5p) Add a new command /threadfun and perform the following:

a. Create 2 threads:

b. The name of the first thread should be "SpawnThread" and it should create 2 additional threads. You can name these two additional threads however you want.

c. The name of the second thread should be "InfoThread" and it must display the following information about every thread in the system: the TID, its Name of the thread, the TID of its Parent and the CPU on which it was spawned (**Hint**: you need to know for each thread on which CPU it was created; use the ApicId field from the CPU structure)

Hint: When creating a thread, you can specify the function that thread will execute as a parameter.

Userprog & Virtual Memory (7p)

1. (1.5p) Implement the basic SyscallVirtualAlloc system call ignoring the FileHandle and Key parameters.

2. (1.5p) Implement a system call SyscallIdMemset which effectively does a memset on a requested virtual address. In the corresponding system call handler check if the pointer receives as a parameter is valid or not.

```
STATUS
SyscallMemset(
    OUT_WRITES(BytesToWrite)    PBYTE   Address,
    IN                          DWORD   BytesToWrite
    IN                          BYTE    ValueToWrite
    );
```

Hint: look how memset is used in other areas of the code

3. (1) Modify Dummy.exe to test ONE of the system calls you created and log relevant information messages in case of success and in case of failure.

Hint: Create your own test cases: choose different values for parameters (both wrong and right values and see if your implementation handles them well).

4. (1p) Log the physical range allocated each time a physical frame is reserved. If the reservation asked, for example, is 3 frames, then something like this should appear in the logs: "Physical range allocated: (0x1000 - 0x3FFF)!\n".

Hint: look for PmmReserveMemoryEx and see what this function returns

5. (2p) Maintain the list of reserved physical frames and their virtual address correspondence for each user process except the System process. For each pair of Virtual Address - Physical Frame, maintain the page access rights as well.

Hint: Add another field for access rights.

NOTES

- Even if not explicitly mentioned, you should not forget we are working in a multi-threading system, synchronization is always necessary when accessing shared structures.
- Even if not explicitly mentioned, you should also **validate user input** when implementing system calls.
- If a problem requires data from previous problem, and you didn't implement the previous problem, you can use dummy values and still get full points.
- If you cannot finish an exercise because it depends on the solution of a previous problem, simply hardcode the needed values.
- Do not forget to **include all the headers you need** in the modified files.

Evaluation criteria (MANDATORY):

- Compilation with no errors in all configurations: "Threads", "Userprog" and "VirtualMemory".
- HAL9000 Boots up (/help command can be executed with success).

A. Threads (4p)

1. 1p
   - correct TID sequence 0.5p
   - log 0.5p
2. 1.5p
   - maintain TID of parent 1p
   - log parent TID 0.5p
3. 1.5p
   - test command added 0.5p
   - first thread created + creates the 5 other threads 0.5p
   - second thread created + displays the required info 0.5p

B. Userprog + Virtual Memory (7p)

1. 1.5p
   - user mode implementation 0.3p
   - kernel mode implementation and call from handler 0.4p
   - parameter validation 0.3p
   - functionality implemented 0.5p
2. 1.5p
   - user mode implementation 0.3p
   - kernel mode implementation and call from handler 0.4p
   - parameter validation 0.3p
   - functionality implemented 0.5p
3. 1p
   - syscall correctly called from Dummy 0.5p
   - test does not fail 0.5p
4. 1p
   - correctly log the start address of the range 0.5p
   - correctly log the end address of the range 0.5p
5. 2p
   - correctly maintain the list of PA and VA pairs for each process (insert and remove elements appropriately) 1.5p
   - maintain the access rights for each pair 0.5p

LAB_EXAM_GRADE = Threads 4p + (Userprog + Virtual Memory) 7p = 11p

Max grade is 11. The minimum grade in order to pass must be 5 (5 points).

MANDATORY: Points(Threads) >= 2 && Points(Userprog + Virtual Memory) >= 3 (**min GRADE 5**)

PAY ATTENTION! Grades lower than 5 will not be rounded to 5 (4.8 will not equal 5).

asdgf.zip

Finish review